

Python语言快速入门

使用空格缩进划分代码块，而非使用大括号和关键词

Python的安装与简单使用

表 1.2 常见库的 pip 安装方法		
库名	库说明	安装方法
NumPy	科学计算和数据分析的基础库	pip install numpy
SciPy	NumPy 基础上的科学计算库	pip install scipy
SymPy	符号计算库	pip install sympy
Pandas	NumPy 基础上的数据分析库	pip install pandas
Matplotlib	数据可视化库	pip install matplotlib
Scikit-learn	机器学习库	pip install scikit-learn
Statsmodels	SciPy 统计函数的补充库	pip install statsmodels
NetworkX	图论和复杂网络库	pip install networkx
cvxpy	凸优化库	pip install 文件名 (离线安装)
TensorFlow	深度学习库	pip install tensorflow
NLTK	自然语言库	pip install nltk
python-louvain	社交网络挖掘的社区发现算法库	pip install python-louvain
PIL	数字图像处理库	pip install pillow
OpenCV	计算机视觉库	pip install opencv-python

```
In [26]: print("hello world!!!")
hello world!!!
```

Python基础知识

基本数据处理

变量命名规则：

- 第一个字符必须是英文字母，下划线“_”或中文，其余字符可以搭配其他的大小写英文字母，数字，下划线或中文
- 不能使用Python内置的关键字
- 区分大小写字母

变量名 = 数据

例如：

```
In [27]: score=100

n=m=20

a,b,c,=80,60,40

a=10;b=20
```

数值数据类型

整数 浮点数

布尔数据类型

Ture False

字符串数据类型

一个字符串可以使用单引号，双引号

三引号通常用于多行文本信息

```
In [28]: str1="hello world"

str2='python'

str3='''hello!

world!'''

print(str1)
print(str2)
print(str3)
# print默认结尾是换行符\n，把end改成空字符串或空格就能同行
print(str1, end=" ")
print(str2, end=" ")
# 一个 print 多参数输出
print(str1, str2, str3)
```

hello world

python

hello!

world!

hello world python hello world python hello!

world!

输出print和输入input

输出print () 函数

```
In [29]: print("四书五经")
print("《大学》", "《中庸》", "《论语》", "《孟子》", sep='、')
print("《大学》", "《中庸》", "《论语》", "《孟子》")
print("《大学》", "《中庸》", "《论语》", "《孟子》", end='    ')
print("《易经》", "《诗经》", "《尚书》", "《礼记》", "《春秋》")
```

四书五经

《大学》、《中庸》、《论语》、《孟子》

《大学》 《中庸》 《论语》 《孟子》

《大学》 《中庸》 《论语》 《孟子》 《易经》 《诗经》 《尚书》 《礼记》 《春秋》

print支持格式化功能，由“%”字符和后面的格式化字符串控制

%s %f %d和c语言一样

```
In [30]: name="张三";age=20
print("%s的年龄是%d"%(name, age))
print("{}的年龄是{}".format(name,age))
```

张三的年龄是20

张三的年龄是20

对齐效果(加负号是左对齐)

```
In [31]: str="hi"
print("%7s"%str)
num=123
print("%7d"%num)
num=123.456
print("%7.2f"%num)
```

hi

123

123.46

输出转义字符

表 1.3 常用的转义字符

转义字符	含义
\t	水平制表字符
\n	换行符
\"	显示双引号
\'	显示单引号
\\	显示反斜杠

输入input () 函数

input输入的是字符串，用int()或eval()变成整数，用float()变成浮点数

```
In [32]: height=input("请输入你的身高： ")
print(height)
```

185

```
In [33]: pi=3.1415926
r=float(input("请输入圆的半径： "))
print("圆的周长是： %.7f" %(2*pi*r))
```

圆的周长是： 12.5663704

四种进制格式输出方式

表 1.4 常用的 4 种进制格式及说明

格式符	说明
%d	输出十进制数
%o	输出八进制数
%x	输出十六进制数, 超过 10 的数字以小写字母表示, 例如 0xff
%X	输出十六进制数, 超过 10 的数字以大写字母表示, 例如 0xFF

运算符与表达式

算术运算符

```
In [34]: x1=58+32
x2=89-28
x3=3*12
x4=125/7    #正常除法, 带小数
x5=125//7   #整除
x6=4**3     #幂次
x7=46%5     #取余数
print(x1,x2,x3,x4,x5,x6,x7,sep=" ")
```

90 61 36 17.857142857142858 17 64 1

复合赋值运算符

跟c语言一样

x+=1

x-=2

关系运算符

成立返回true, 不成立返回false

跟c语言一样

表 1.5 关系运算结果

运算符	说明
>	a 大于 b, 返回 True
<	a 小于 b, 返回 False
>=	a 大于或等于 b, 返回 True
<=	a 小于或等于 b, 返回 False
==	a 等于 b, 返回 True
!=	a 不等于 b, 返回 True

逻辑运算符

not, and, or

```
In [35]: print(100>2)and(52>41)
total=124
```

```
value=(total%4==0)and(total%3==0)
print(value)
```

True
False

位运算符

表 1.6 二进制数字操作的按位运算符

运算符	描述	实例
~	按位取反 即 $\sim x = -(x + 1)$	$\sim 5 = -6$
<<	按位左移	$5 << 2 = 20$, 将 101 向左移动两位, 得 10100
>>	按位右移	$5 >> 2 = 1$, 将 101 向右移动两位并去掉小数部分, 得 1
&	按位与	$5 \& 3 = 1$, 将对应的二进制数执行按位与操作, 即 $101 \& 011 = 001 = 1$
	按位或	$5 3 = 7$, 将对应的二进制数执行按位或操作, 即 $101 011 = 111 = 7$
^	按位异或	$5 \wedge 3 = 6$, 将对应的二进制数执行按位异或操作 (对位相加, 不进位), 即 $101 \wedge 011 = 110 = 6$

转二进制函数bin ()

```
In [36]: print(bin(10))
```

0b1010

流程控制

3种流程控制结构, 即if, for, while

if语句

if, elif, else

注意if后要加冒号: , 语句要缩进

```
In [37]: a,b=eval(input("请输入两个整数: "))
if a>=b:
    print("最大的数为:%d"%a)
else:
    print("最大的数为:%d"%b)
```

Traceback (most recent call last):

File ~\AppData\Roaming\Python\Python314\site-packages\IPython\core\interactiveshell.py:3748 in run_code
exec(code_obj, self.user_global_ns, self.user_ns)

Cell In[37], line 1
a,b=eval(input("请输入两个整数: "))

File <string>:1
10 5
^

SyntaxError: invalid syntax

for循环

配合range () 函数使用，返回一个等差数列的可迭代对象，初始值默认为0，步长默认为1，要转化成列表

```
In [ ]: print(list(range(5)))
        print(list(range(1,6)))
        print(list(range(1,10,2)))
```

[0, 1, 2, 3, 4]

[1, 2, 3, 4, 5]

[1, 3, 5, 7, 9]

```
In [ ]: sum=0
        number=eval(input("请输入一个整数："))
        print("从小到大排列输出数字：")
        for i in range(1,number+1):
            sum+=i
            print("%d"%i,end="")
            if i<number:
                print("+",end="")
            else :
                print("=",end="")
        print("%d"%sum)
        print("从大到小排列输出数字：")
        for i in range(number,0,-1):
            print("%d"%i,end="")
            if i>1:
                print("+",end="")
            else :
                print("=",end="")
        print("%d"%sum)
```

从小到大排列输出数字：

1+2+3+4+5+6+7=28

从大到小排列输出数字：

7+6+5+4+3+2+1=28

while循环

例：求 $\sin x = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n+1}}{(2n+1)!} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$ 直到最后一项小于 10^{-6}

时停止计算

```
In [ ]: from math import *
        n=0;x1=float(input("请输入一个角度"))
        x=radians(x1)
        s=a=x
        while abs(a)>1e-6:
            a*=(-1)*x*x/(2*n+3)/(2*n+2)
            n+=1
            s+=a
        print("x={},sin(x)={}".format(x1,s))
```

x=32.0,sin(x)=0.529919264274441

复合数据类型

列表，元组，集合，字典

列表，元组，字符串有序，称为序列，可以用索引访问

集合，字典无顺序

list列表

索引

正向索引，负向索引，切片索引，无限索引

```
In [ ]: a=["张三","男","23","江苏","硕士","已婚",["身高172","体重70"]]
print(a[0])
print(a[-1])
print(a[-1][1])
print(a[-3:])
print(a[:3])
print(a[:2])
print(a[0:-1])
```

张三

['身高172', '体重70']

体重70

['硕士', '已婚', ['身高172', '体重70']]

['张三', '男', '23']

['张三', '23', '硕士', ['身高172', '体重70']]

['张三', '男', '23', '江苏', '硕士', '已婚']

列表元素的增加

append, extend, insert

```
In [ ]: a=[1,2,3]
a.append(4)
print(a)
a.extend([3,2,1])
print(a)
a.insert(0,-1)
print(a)
```

[1, 2, 3, 4]

[1, 2, 3, 4, 3, 2, 1]

[-1, 1, 2, 3, 4, 3, 2, 1]

列表元素的删除

pop,remove,clear

```
In [ ]: a=[10,20,30,5,8,5,8,6,6]
a.pop() #尾删
print(a)
a.pop(2) #按索引删除
print(a)
a.remove(8);a.remove(8) #从左删第一个
print(a)
a.clear()
print(a)
```

```
[10, 20, 30, 5, 8, 5, 8, 6]
[10, 20, 5, 8, 5, 8, 6]
[10, 20, 5, 5, 6]
[]
```

列表的常用操作

表 1.7 Python 中列表的常用操作 (其中 a = [1, 2, 3, 4], b = [7, 6, 5])			
名称	操作	说明	实例结果
查找元素下标	a.index(e)	查找元素 e 在列表 a 中的下标位置, 如果没有找到则报错	a.index(3), 输出 2
计算元素次数	a.count(e)	计算元素 e 在列表 a 中出现的次数, 如果没有出现则返回 0	a.count(2), 输出 1
连接两个列表	a+b	将列表 b 中各元素依次插入列表 a 尾部, 并返回一个新的列表	a+b, 输出 [1,2,3,4,7,6,5]
列表重复	b*n	列表 b 重复 n 次	b*2, 输出 [7,6,5,7,6,5]
列表长度	len(a)	获取列表 a 的长度, 即列表中元素个数	len(a), 输出 4
排序	b.sort()	一般根据同类元素列表的数据类型, 对列表中各元素按“升序”排列. 该函数改变原列表, 且无返回值	b.sort, 输出 [5,6,7]
排序	c=sorted(b)	一般根据同类元素列表的数据类型, 对列表中各元素按“升序”排列, 得到新列表	sorted(b), 输出 [5,6,7]
排序	c=sorted(b,reverse=True)	一般根据同类元素列表的数据类型, 对列表中各元素按“降序”排列, 得到新列表	sorted(b), 输出 [7,6,5]
反转	a.reverse()	按列表中元素的下标位置逆序输出各元素	a.reverse(), 输出 [4,3,2,1]

tuple元组, dict字典, set集合

tuple元组

不能被修改的列表, 可以只用逗号分隔, 或用小括号, 只有count, index

```
In [ ]: my_tuple=(1,2,3,4,5)
print(my_tuple)
tuple1=1,2,3,4,5
print(tuple1)
tuple2=1,
print(tuple2)
```

```
(1, 2, 3, 4, 5)
(1, 2, 3, 4, 5)
(1,)
```

dic字典

- 大括号, 键值对, key: value
- 键要唯一
- 字典不再是序列, 不能位置索引, 只能键索引
- 可变
- 可以用dict () 函数先创建空字典

```
In [ ]: d={"姓名":"张三","年龄":"33","子女":{"子":"张四","女":"张玲"}}
print(d)
print(d["年龄"])
d["年龄"]=35
```



```
print(d)
d["户籍"]="烟台"
print(d)
d.pop("子女")
print(d)
```

```
{'姓名': '张三', '年龄': '33', '子女': {'子': '张四', '女': '张玲'}}
33
{'姓名': '张三', '年龄': 35, '子女': {'子': '张四', '女': '张玲'}}
{'姓名': '张三', '年龄': 35, '子女': {'子': '张四', '女': '张玲'}, '户籍': '烟台'}
{'姓名': '张三', '年龄': 35, '户籍': '烟台'}
```

集合

有互异性

集合的创建

- 大括号和逗号分割
- `set()` 函数，将字符串，列表，元组转换成集合类型的数据

```
In [ ]: a=set("abcde")
print(a)
b=[1,2,2,2,3,5,6,6]
c=set(b)      #转化为集合，去除重复元素
print(list(c))
```

```
{'e', 'a', 'b', 'c', 'd'}
[1, 2, 3, 5, 6]
```

集合的常用方法

表 1.8 集合的常用方法

方法	意义
<code>s.add(x)</code>	在集合 <code>s</code> 中添加对象 <code>x</code> ，如果对象已经存在，则不添加
<code>s.remove(x)</code>	从集合 <code>s</code> 中删除 <code>x</code> ，若 <code>x</code> 不存在，则引发 <code>KeyError</code> 错误
<code>s.discard(x)</code>	如果 <code>x</code> 是 <code>s</code> 的成员，则删除 <code>x</code> 。 <code>x</code> 不存在，也不出现错误
<code>s.clear()</code>	清空集合 <code>s</code> 中所有元素
<code>s.copy()</code>	将集合 <code>s</code> 进行一次浅拷贝
<code>s.pop()</code>	从集合 <code>s</code> 中删除第一个元素，如果 <code>s</code> 为空，则引发 <code>KeyError</code> 异常
<code>s.update(s2)</code>	用 <code>s</code> 与 <code>s2</code> 得到的并集更新变量 <code>s</code>

表 1.9 用运算符操作实现的方法

方法	意义
<code>s.difference(s2)</code>	集合的差，等同于 $s = s - s2$
<code>s.intersection(s2)</code>	集合的交，等同于 $s = s \& s2$
<code>s.symmetric_difference(s2)</code>	集合的对称差，等同于 $s = s \wedge s2$
<code>s.union(s2)</code>	集合的并集，等同于 $s = s s2$

序列的一些实用操作

string字符串的实用操作

`str()`：将数值数据转化成字符串

`+`：拼接字符串

len():字符串长度

count():字符串子字符串的出现次数

eval():把字符串变成Python语句执行

find():返回子字符串的左端索引, 没找到返回-1

rfind():从右往左查找

split():将字符串分割成序列, 默认按空格分隔

```
In [ ]: str1="I am a student"
list1=str1.split()
str2="1,2,3,4"
list2=str2.split(",")
print(list1,list2)
```

```
['I', 'am', 'a', 'student'] ['1', '2', '3', '4']
```

strip():去除字符串开头结尾的空格字符, lstrip():去除开头空格, rstrip():去除结尾空格

```
In [ ]: str="    I am a student    "
print(str.strip())
```

```
I am a student
```

join():可认为是split()函数的逆方法

几个序列操作函数

lambda函数, 也称为匿名函数

例: $f(x, y) = |x| + y^3$

```
In [ ]: f=lambda x,y:abs(x)+y**3
print(f(-3,2))
```

```
11
```

map()

调用格式map(func,*iterables)

map()接收一个函数func和一个列表, 把func依次作用在列表的每个元素上, 得到一个新列表

```
In [ ]: a=map(lambda x:pow(x,2),[1,2,3,4])
# a=map(pow,range(6),[2 for b in range(6)])
print(list(a))
```

```
[0, 1, 4, 9, 16, 25]
```

reduce()

调用格式reduce(function,sequence[,initial])

function是有两个参数的函数,sequence是元组, 列表, 字典, 字符串等可迭代的对象, initial是可选的初始值

```
In [ ]: from functools import reduce
print(reduce(lambda x,y:x*y,range(1,6)))
print(reduce(lambda x,y:x+y,range(1,7)))
```

120

21

filter()

调用格式filter(function of None,iterable)

对使function返回true的元素经行过滤

```
In [ ]: print(list(filter(lambda x:x%3==0,range(1,21))))
```

[3, 6, 9, 12, 15, 18]

zip()

将多个列表或元组的对应位置的元素组合为元组

```
In [ ]: a=range(1,5)
b=range(5,9)
d=range(9,13)
c=zip(a,b)
print(list(c))
# *操作符将元组分为了两个独立的参数进行传递
print(list(zip(*zip(a,b,d))))
```

[(1, 5), (2, 6), (3, 7), (4, 8)]

[(1, 2, 3, 4), (5, 6, 7, 8), (9, 10, 11, 12)]

enumerate()

枚举

```
In [ ]: a=[(1,5),(2,6),(3,7),(4,8)]
for b in enumerate(a):
    print(b)
```

(0, (1, 5))

(1, (2, 6))

(2, (3, 7))

(3, (4, 8))

列表推导式和元组生成器推导式

列表推导式

```
In [ ]: b=[[0]*6 for i in range(4)]
print(b)
```

[[0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0]]

元组生成器推导式

```
In [ ]: g=((i+1)**2 for i in range(6))
print(tuple(g))
```

(1, 4, 9, 16, 25, 36)

函数

内置函数，第三方函数，自定义函数

自定义函数语法

def functionName(formalParameters):

functionBody

用三引号注释来说明函数

函数体内容不可为空，用pass表示空语句

```
In [ ]: from Pexl_5_1 import *  
print(factorial(5))
```

120

自定义函数的四种参数

位置参数

默认参数

赋了各自的初值

```
In [ ]: def square_sum(n,p=2):  
        result=sum([i**p for i in range(1,n+1)])  
        return (n,p,result)  
print("从1到%d的%d次方和为%d"%square_sum(10))  
print("从1到%d的%d次方和为%d"%square_sum(10,3))
```

从1到10的2次方和为385

从1到10的3次方和为3025

可变参数

不确定该给自定义函数传入多少个参数值,可变参数调用时自动组装为一个元祖

```
In [ ]: def add(*args):  
        print(args,end=" ")  
        s=sum(args)  
        return s  
print("的和为",add(10,12,8,6))
```

(10, 12, 8, 6) 的和为 36

关键字函数

关键字参数会把带参数名的参数值组装到一个字典中，键就是具体的实参名，值就是传入的参数值

```
In [ ]: def person(name,age,**kw):
        print("name:",name,"age:",age,"other:",kw)

        person("Michael",30)
        person("Bob",35,cith="Beijing")
        person("Adam",45,gender="M",job="Engineer")
```

```
name: Michael age: 30 other: {}
name: Bob age: 35 other: {'cith': 'Beijing'}
name: Adam age: 45 other: {'gender': 'M', 'job': 'Engineer'}
```

参数传递

参数传递方式

- 传值：函数中对参数值的修改，不影响原来的自变量值（不可变对象，如数值，字符串）
- 传址：参数值的变动，会影响原来的自变量值（可变对象，如列表）

```
In [ ]: # 传值调用
def fun(a,b):
    a,b=b,a
    print("函数内交换数值后: a=%d,\tb=%d"%(a,b))

a=10;b=15
print("调用函数前的数值: a=%d,\tb=%d"%(a,b))
print("-"*40)
fun(a,b)
print("-"*40)
print("调用函数后的数值: a=%d,\tb=%d"%(a,b))
```

调用函数前的数值: a=10, b=15

函数内交换数值后: a=15, b=10

调用函数后的数值: a=10, b=15

```
In [ ]: # 传址调用
def change(data):
    data[0],data[1]=data[1],data[0]
    print("函数内交换数值后: ",end=" ")
    for i in range (2):
        print("data[%d]=%d"%(i,data[i]),end="\t")

data=[16,25]
print("原始数据是:",end=" ")
for i in range (2):
    print("data[%d]=%d"%(i,data[i]),end="\t")
print("\n","-"*40)
change(data)
print("\n","-"*40)
print("排序后的数据: ",end=" ")
for i in range (2):
    print("data[%d]=%d"%(i,data[i]),end="\t")
```

原始数据是: data[0]=16 data[1]=25

函数内交换数值后: data[0]=25 data[1]=16

排序后的数据: data[0]=25 data[1]=16

参数传递时的复合数据解包

传递参数时, 可以使用列表, 元组, 集合, 字典以及其他可迭代对象作为实参, 并在实参前加一个星号, Python解释器将自动进行解包, 然后传递给多个单变量形参

需要注意的是, 如果使用字典作为形参, 则默认使用字典的键, 如果需要将字典中的键值对作为参数则需要使用items()方法

最后, 请保证实参元素个数与形参个数相等

```
In [ ]: def fun(a,b,c):  
        print("三个数的和是: ",a+b+c)  
        seq=[1,2,3];fun(*seq)  
        tup=(4,5,6);fun(*tup)  
        dic={1:"a",2:"b",3:"c"};fun(*dic)  
        set={7,8,9};fun(*set)
```

三个数的和是: 6

三个数的和是: 15

三个数的和是: 6

三个数的和是: 24

两个特殊函数

匿名函数

没有函数名的简单函数, 只可以包含一个表达式, 结果就是函数的返回值

```
In [ ]: f=lambda a,b=2,c=5:a-b+c  
        print("f=",f(10,20)) #默认值参数  
        print("f=",f(10,20,30))  
        print("f=",f(c=20,a=10,b=30)) #使用关键字实参
```

f= -5

f= 20

f= 0

例: 先定义函数求 $\sum_{k=1}^n k^m$, 然后调用该函数求 $s = \sum_{k=1}^{100} k + \sum_{k=1}^{50} k^2 + \sum_{k=1}^{10} \frac{1}{k}$

```
In [ ]: f=lambda m,n:sum([ i**m for i in range(1,n+1)])  
        s=f(1,100)+f(2,50)+f(-1,10)  
        print("s=",s)
```

s= 47977.92896825397

递归函数

函数的函数体中又直接或间接地调用该函数本身

```
In [ ]: # n=int(input("请输入n的值: "))  
        n=8
```

```
def fac(n):
    if n<=1:
        return 1
    else:return n*fac(n-1)
print("%d!=%d"%(n,fac(n)))
```

8!=40320

例:用递归方法计算下列多项式函数的值.

$$p(x, n) = x - x^2 + x^3 - x^4 + \cdots + (-1)^{n-1} x^n \quad (n > 0).$$

```
In [ ]: def p(x,n):
        if n==1:
            return x
        else :
            return x*(1-p(x,n-1))

x,n=2,4
print("p(%d,%d)=%d"%(x,n,p(x,n)))
```

p(2,4)=-10

导入模块

时间相关的time, datetime模块, 随机数的random模块, 与操作系统交互的os模块, 对python解释器相关的sys模块, 数学计算的math模块 等十几个模块

查看所有模块: help("modules")

查看math模块的帮助: import math; help(math)

查看math模块的所有函数: import math; dir(math)

导入模块有四种方式:

- import 模块名 [as 别名]
- from 模块名 import 对象名 [as 别名]
- from 模块名 import * (导入模块中通过__all__变量指定的所有对象)
- 自定义模块的导入

例: 求 $y = e^2 + \sum_{n=1}^{100} \frac{1 + \ln n + \sin n}{2\pi}$.

```
In [ ]: import math
f=lambda n:sum([(1+math.log(i)+math.sin(i))/2/math.pi) for i in range(1,101)])
print("sum=%f"%(math.e*math.e+f(n)))
```

sum=81.175230

例:创建函数集合 $f(x) = x^2 + x + 1$, $g(x) = x^3 + 2x + 1$ 和 $h(x) = \frac{1}{f(x)}$ 的自

定义 FunctionSet.py 模块, 调用该模块计算 $f(1)$, $g(2)$ 和 $h(3)$ 的值。

```
In [ ]: import FunctionSet as fs
print("f(1)=", fs.f(1))
print("g(2)=", fs.g(2))
print("h(3)=", fs.h(3))
```

```
f(1)= 3
g(2)= 13
h(3)= 0.07692307692307693
```

Python程序的书写规则

- 满足编程语言的语法规则
- 符合阅读程序的审美规则

语法规则

缩进，冒号，空行

注释

单行注释，多行注释

语句行等其他事项